

Arquitetura de Compiladores e Interpretadores

Disciplina: Compiladores e Interpretadores

Projeto integrador: linguagem MINIC

Linguagens de programação são notações para se descrever computações para pessoas e para máquinas. O mundo conforme o conhecemos depende de linguagens de programação, pois todo o software executando em todos os computadores foi escrito em alguma linguagem de programação. Mas, antes que possa rodar, um programa primeiro precisa ser traduzido para um formato que lhe permita ser executado por um computador.

Os sistemas de software que fazem essa tradução são denominados *compiladores*.

Este livro ensina como projetar e implementar compiladores. Vamos descobrir que algumas poucas idéias básicas podem ser utilizadas na construção de tradutores para uma grande variedade de linguagens e máquinas. Além dos compiladores, os princípios e técnicas para o seu projeto se aplicam a vários outros domínios que provavelmente serão reutilizados muitas vezes na carreira de um cientista da computação. O estudo da escrita de compiladores abrange linguagens de programação, arquitetura de máquina, teoria de linguagem, algoritmos e engenharia de software.

Neste capítulo preliminar, introduzimos as diferentes formas de tradutores de linguagens, apresentamos uma visão de alto nível da estrutura de um compilador típico e discutimos as novas tendências das linguagens de programação e das arquiteturas de máquina que estão influenciando os compiladores. Incluímos algumas observações sobre o relacionamento entre o projeto de um compilador e a teoria da ciência da computação, além de um esboço das aplicações da tecnologia de compilador que ultrapassam a fronteira da compilação. Terminamos com uma rápida apresentação dos principais conceitos de linguagem de programação que serão necessários para o nosso estudo dos compiladores.

1.1 PROCESSADORES DE LINGUAGEM

Colocando de uma forma bem simples, um compilador é um programa que recebe como entrada um programa em uma linguagem de programação - a linguagem *fonte* - e o traduz para um programa equivalente em outra linguagem - a linguagem *objeto*; ver Figura 1.1. Um papel importante do compilador é relatar quaisquer erros no programa fonte detectados durante esse processo de tradução.

programa fonte
|

Figura de abertura: visão caixa-preta de um compilador. Fonte: Aho et al., Cap. 1, Fig. 1.1, p. 1.

Objetivo da aula: compreender como programas são processados e como as fases de um compilador se organizam, distinguindo compilação, interpretação e modelos híbridos.

1. Objetivos de aprendizagem

Ao final desta aula, você deverá ser capaz de:

- explicar o papel de um compilador como tradutor entre linguagens;
- diferenciar compilador, interpretador e sistema híbrido;
- identificar as partes de um compilador: análise, síntese e otimização;
- descrever o papel da tabela de símbolos e das representações intermediárias;
- relacionar cada fase ao projeto do compilador MINIC.

2. Processadores de linguagem

Um processador de linguagem transforma ou executa programas escritos em uma linguagem de programação. O compilador recebe um programa-fonte e produz um programa equivalente em uma linguagem-objeto. Durante o processo, também deve detectar erros e fornecer diagnósticos úteis.

Linguagens de programação são notações para se descrever computações para pessoas e para máquinas. O mundo conforme o conhecemos depende de linguagens de programação, pois todo o software executando em todos os computadores foi escrito em alguma linguagem de programação. Mas, antes que possa rodar, um programa primeiro precisa ser traduzido para um formato que lhe permita ser executado por um computador.

Os sistemas de software que fazem essa tradução são denominados *compiladores*.

Este livro ensina como projetar e implementar compiladores. Vamos descobrir que algumas poucas idéias básicas podem ser utilizadas na construção de tradutores para uma grande variedade de linguagens e máquinas. Além dos compiladores, os princípios e técnicas para o seu projeto se aplicam a vários outros domínios que provavelmente serão reutilizados muitas vezes na carreira de um cientista da computação. O estudo da escrita de compiladores abrange linguagens de programação, arquitetura de máquina, teoria de linguagem, algoritmos e engenharia de software.

Neste capítulo preliminar, introduzimos as diferentes formas de tradutores de linguagens, apresentamos uma visão de alto nível da estrutura de um compilador típico e discutimos as novas tendências das linguagens de programação e das arquiteturas de máquina que estão influenciando os compiladores. Incluímos algumas observações sobre o relacionamento entre o projeto de um compilador e a teoria da ciência da computação, além de um esboço das aplicações da tecnologia de compilador que ultrapassam a fronteira da compilação. Terminamos com uma rápida apresentação dos principais conceitos de linguagem de programação que serão necessários para o nosso estudo dos compiladores.

1.1 PROCESSADORES DE LINGUAGEM

Colocando de uma forma bem simples, um compilador é um programa que recebe como entrada um programa em uma linguagem de programação - a linguagem *fonte* - e o traduz para um programa equivalente em outra linguagem - a linguagem *objeto*; ver Figura 1.1. Um papel importante do compilador é relatar quaisquer erros no programa fonte detectados durante esse processo de tradução.

programa fonte
↓

Fig. 1 — Compilador como tradutor de programa-fonte para programa-objeto. Fonte: Aho et al., Fig. 1.1, p. 1.

Conceito-chave: equivalência semântica significa que, para entradas compatíveis, o programa traduzido deve preservar o comportamento definido pelo programa original.

3. Compilador, interpretador e modelo híbrido

No modelo compilado, a tradução ocorre antes da execução. No modelo interpretado, um programa processa as operações do código-fonte durante a execução. Um sistema híbrido combina as duas estratégias: o código-fonte é convertido para uma representação intermediária, como bytecode, que pode ser executada por uma máquina virtual.

2

COMPILADORES: PRINCÍPIOS, TÉCNICAS E FERRAMENTAS

Um *interpretador* é outro tipo comum de processador de linguagem. Em vez de produzir um programa objeto como resultado da tradução, um interpretador executa diretamente as operações especificadas no programa fonte sobre as entradas fornecidas pelo usuário, como mostra a Figura 1.3.

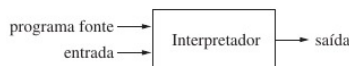


FIGURA 1.3 Um interpretador.

O programa objeto em linguagem de máquina produzido por um compilador normalmente é muito mais rápido no mapeamento de entradas para saídas do que um interpretador. Porém, um interpretador frequentemente oferece um melhor diagnóstico de erro do que um compilador, pois executa o programa fonte instrução por instrução.

EXEMPLO 1.1: Os processadores da linguagem Java combinam compilação e interpretação, como mostrado na Figura 1.4.

Fig. 2 — Processamento interpretado. Fonte: Aho et al., Fig. 1.3, p. 2.

EXEMPLO 1.1: Os processadores da linguagem Java combinam compilação e interpretação, como mostrado na Figura 1.4. Um programa fonte em Java pode ser primeiro compilado para uma forma intermediária, chamada *bytecodes*. Os bytecodes (ou códigos de bytes) são então interpretados por uma máquina virtual. Como um benefício dessa combinação, os bytecodes compilados em uma máquina podem ser interpretados em outra máquina, talvez por meio de uma rede.

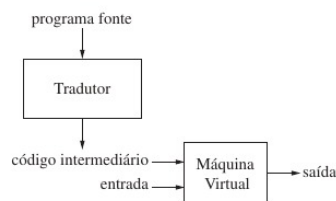


FIGURA 1.4 Um compilador híbrido.

A fim de conseguir um processamento mais rápido das entradas para as saídas, alguns compiladores Java, chamados compiladores *just-in-time*, traduzem os bytecodes para uma dada linguagem de máquina imediatamente antes de executarem o programa intermediário para processar a entrada.

Além de um compilador, vários outros programas podem ser necessários para a criação de um programa objeto executável, como mostra a Figura 1.5. Um programa fonte pode ser subdividido em módulos armazenados em arquivos separados. A tarefa de coletar o programa fonte às vezes é confiada a um programa separado, chamado *pré-processador*. O pré-processador também pode expandir *macros* em comandos na linguagem fonte.

Fig. 3 — Modelo híbrido com código intermediário e máquina virtual. Fonte: Aho et al., Fig. 1.4, p. 2.

Comparação: código nativo costuma oferecer maior desempenho após a compilação; interpretação favorece portabilidade e pode simplificar o diagnóstico em tempo de execução; modelos híbridos equilibram portabilidade e desempenho.

4. O sistema de processamento de linguagem

A criação de um programa executável pode envolver mais componentes além do compilador. Um pré-processador modifica o código-fonte; o compilador pode gerar assembly; o montador converte assembly em código de máquina relocável; o editor de ligação combina módulos e bibliotecas; e o carregador prepara o programa para execução.

vel, como mostra a Figura 1.5. Um programa fonte pode ser subdividido em módulos armazenados em arquivos separados. A tarefa de coletar o programa fonte às vezes é confiada a um programa separado, chamado *pré-processador*. O pré-processador também pode expandir *macros* em comandos na linguagem fonte.

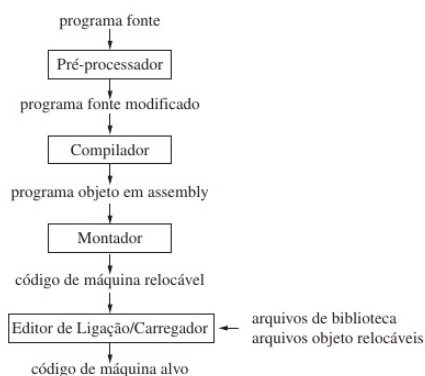


FIGURA 1.5 Um sistema de processamento de linguagem.

Fig. 4 — Sistema de processamento de linguagem. Fonte: Aho et al., Fig. 1.5, p. 2.

Esse fluxo é importante para entender que “compilar” não é necessariamente uma única operação. Em projetos reais, várias ferramentas cooperam para transformar o código escrito pelo programador em um processo executável.

5. Análise e síntese: front-end e back-end

A arquitetura de um compilador pode ser compreendida em duas grandes partes: **análise** e **síntese**. A análise examina o programa-fonte, identifica sua estrutura e produz uma representação intermediária. A síntese usa essa representação e as informações da tabela de símbolos para construir o programa-objeto. Essas partes são frequentemente chamadas de **front-end** e **back-end**, respectivamente.

6. Fases de um compilador

As fases são organizadas como transformações sucessivas. O fluxo típico começa com caracteres e termina com código da máquina-alvo. A tabela de símbolos acompanha o processo, armazenando informações sobre identificadores, tipos, escopos e outras propriedades.

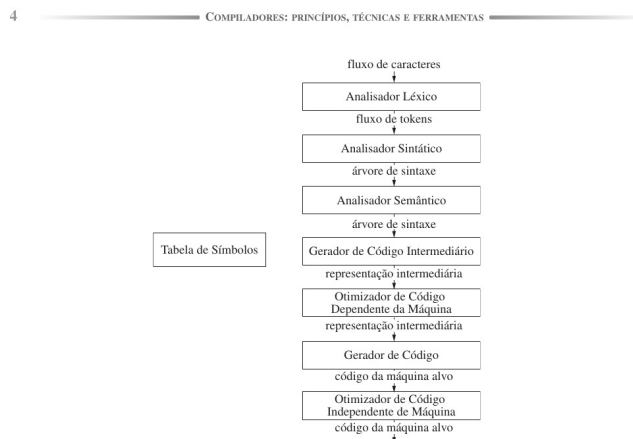


FIGURA 1.6 Fases de um compilador.

Por exemplo, suponha que um programa fonte contenha o comando de atribuição

`position = initial + rate * 60` (1.1)

Os caracteres nessa atribuição poderiam ser agrupados nos seguintes lexemas e mapeados para os seguintes tokens passados ao analisador sintático:

1. `position` é um lexema mapeado em um token $\langle id, 1 \rangle$, onde `id` é um símbolo abstrato que significa *identificador* e 1 aponta para a entrada da tabela de símbolos onde se encontra `position`. A entrada da tabela de símbolos para um identificador mantém informações sobre o identificador, como seu nome e tipo.
2. O símbolo de atribuição `=` é um lexema mapeado para o token $\langle = \rangle$. Como esse token não precisa de um valor de atributo, omitimos o segundo componente. Poderíamos ter usado qualquer símbolo abstrato, como **atribuir** para o nome do token, mas, por conveniência de notação, escolhemos usar o próprio lexema como nome do símbolo abstrato.
3. `initial` é um lexema mapeado para o token $\langle id, 2 \rangle$, onde 2 aponta para a entrada da tabela de símbolos onde se encontra `initial`.
4. `+` é um lexema mapeado para o token $\langle + \rangle$.
5. `rate` é um lexema mapeado para o token $\langle id, 3 \rangle$, onde o valor 3 aponta para a entrada da tabela de símbolos onde se encontra `rate`.
6. `*` é um lexema mapeado para o token $\langle * \rangle$.
7. `60` é um lexema mapeado para o token $\langle 60 \rangle$.

Os espaços que separam os lexemas são descartados pelo analisador léxico.

Fig. 5 — Fases de um compilador. Fonte: Aho et al., Fig. 1.6, p. 4.

As fases podem ser agrupadas em implementações reais, mas a decomposição é didaticamente útil porque permite atribuir responsabilidades claras a cada componente:

Fase	Responsabilidade
Análise léxica	Agrupar caracteres em lexemas e produzir tokens.
Análise sintática	Verificar a estrutura gramatical e construir uma árvore.
Análise semântica	Verificar tipos, escopos e regras de significado.
Código intermediário	Produzir uma representação independente da máquina.
Otimização	Melhorar a representação preservando o comportamento.
Geração de código	Produzir código para a máquina-alvo.

7. Exemplo integrado: uma atribuição

Considere a instrução MINIC:

```
position = initial + rate * 60;
```

O analisador léxico identifica identificadores, operador de atribuição, operadores aritméticos e literal numérico. O parser usa esses tokens para construir uma árvore que respeita a precedência: a multiplicação é agrupada antes da adição. A análise semântica consulta a tabela de símbolos para verificar se os identificadores existem e se os tipos são compatíveis. Uma representação intermediária possível é:

```
t1 = rate * 60
t2 = initial + t1
position = t2
```

Depois, o back-end escolhe instruções e locais de armazenamento para gerar o código-alvo. O exemplo mostra por que as fases precisam compartilhar representações e informações: cada etapa depende do resultado estruturado da anterior.

8. Aplicação ao projeto MINIC

Componente	Pergunta orientadora
Scanner	Quais sequências de caracteres formam tokens MINIC?
Parser e AST	Como os tokens formam declarações, expressões e comandos?
Semântica e IR	Quais tipos, escopos e instruções intermediárias são necessários?
Back-end	Qual código-alvo será gerado e como será otimizado?

Atividade em equipe

Desenhe o pipeline do MINIC para o programa `int main() { print(2 + 3 * 4); return 0; }`. Indique: (a) tokens; (b) estrutura sintática; (c) verificações semânticas; (d) uma IR possível; e (e) a diferença entre executar a AST por interpretação e gerar código para uma máquina-alvo.

9. Síntese e exercícios

Exercício 1. Explique, com suas palavras, a diferença entre compilador e interpretador. Cite uma vantagem e uma limitação de cada estratégia.

Exercício 2. Diferencie front-end e back-end. Em qual parte se encontram a análise sintática e a seleção de instruções?

Exercício 3. Complete o fluxo: caracteres → _____ → _____ → representação intermediária → _____.

Exercício 4. Para a expressão $x = a + b * c$; , explique por que a árvore sintática não deve agrupar a soma antes da multiplicação.

Exercício 5. Relacione cada informação à estrutura que normalmente a armazena: nome de variável, tipo, escopo, temporário e rótulo.

Leitura recomendada

Aho, Lam, Sethi e Ullman. *Compiladores: Princípios, Técnicas e Ferramentas*. Capítulo 1: Introdução, especialmente as seções 1.1 — Processadores de linguagem e 1.2 — A estrutura de um compilador. Na edição consultada: pp. 1–8; as figuras utilizadas nesta apostila estão nas pp. 1–4.

Nota sobre as imagens

As figuras desta apostila são reproduzidas em escala reduzida exclusivamente para fins didáticos, com indicação da obra e da página de origem. O conteúdo textual foi redigido como material de apoio e não substitui a leitura do livro.